

University of Nottingham (nottingham-ac) - Developer Guide

Date: September 23, 2026

Adobe

Table of Contents

University of Nottingham (nottingham-ac) - Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development Setup	6
Design System	7
Blocks Reference	8
Universal Editor Models	10
Common Development Tasks	10
Environments	11
Git Workflow	11
Troubleshooting	11
Resources	11

University of Nottingham (nottingham-ac) - Developer Guide

This is the technical guide for developers taking over the **nottingham-ac** site — a migration of nottingham.ac.uk onto Adobe Experience Manager Edge Delivery Services. It reflects the current state of the codebase and supersedes any prior version.

Quick Reference

Resource	URL
Code Repository	polizzigaetano/nottingham-ac (GitHub — from Config Service)
Config Service Org	polizzigaetano
Preview	https://main-nottingham-ac-polizzigaetano.aem.page/
Live	https://main-nottingham-ac-polizzigaetano.aem.live/
Local Dev	http://localhost:3000
Content Source	AEM as a Cloud Service author (author-p151992-e1858597.adobecloud.com), markup delivery

Setup type: This is a **repoless / multi-site** Config Service organization — [polizzigaetano](https://github.com/polizzigaetano) hosts many sites (aemdocb, citisignal, edsdemo, premierinn, and others); [nottingham-ac](https://github.com/polizzigaetano/nottingham-ac) is one of them. The code for this site lives in the GitHub repo [polizzigaetano/nottingham-ac](https://github.com/polizzigaetano/nottingham-ac) and content is served from an AEM Cloud Service author instance. This matters for local development (see Local Development Setup).

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+) — no transpiling, no build step
- CSS3 with Custom Properties (design tokens)
- HTML5 semantic markup generated by the aem.live backend, decorated client-side
- Content authored in AEM (Universal Editor / crosswalk [xwalk](https://github.com/adobe/crosswalk) project), delivered as markup
- Based on the [aem-boilerplate-xwalk](https://github.com/adobe/aem-boilerplate-xwalk) boilerplate

Project Structure

```
├── blocks/           # 27 block folders (see Blocks Reference)
├── models/           # Universal Editor component definitions/models/filters (partials)
├── scripts/
│   ├── aem.js        # Core library (DO NOT MODIFY)
│   └── scripts.js     # Custom page decoration + Dynamic Media auto-block + button
├── decoration
│   └── delayed.js    # Delayed phase (currently empty – no third-party scripts)
```

```

├── editor-support.js / editor-support-rte.js # Universal Editor authoring support
├── dompurify.min.js # HTML sanitisation dependency
├── styles/
│   ├── brand.css # Reconciled University of Nottingham design tokens (imported first)
│   ├── styles.css # Critical styles + semantic tokens + button system
│   ├── fonts.css # @font-face declarations (loaded lazily)
│   └── lazy-styles.css # Non-critical global styles
├── icons/ # SVG icons
└── stardust/ # Design-extraction provenance artifacts (not served)

```

Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. The orchestration lives in `scripts/scripts.js` (`loadPage` → `loadEager` → `loadLazy` → `loadDelayed`).

Phase	Purpose	What Loads	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	<code>styles/styles.css</code> (which <code>@imports</code> <code>brand.css</code>), first section's blocks, <code>scripts/scripts.js</code> ; on desktop (<code>innerWidth >= 900</code>) or when previously cached, <code>fonts.css</code> is pulled early	Blocks LCP — kept minimal
Lazy	Load remaining page content after first paint	Remaining sections/blocks, header, footer, <code>fonts.css</code> , <code>lazy-styles.css</code>	Runs after Eager completes
Delayed	Load non-essential third-party scripts ~3s later	<code>scripts/delayed.js</code> (currently empty)	Never blocks rendering

Rules for developers: - Never add third-party scripts to `scripts.js` — they block LCP. Use `delayed.js`. - Fonts load lazily by default; the eager phase only pre-loads them on fast (desktop) connections or when a `fonts-loaded` session flag is set. - Blocks in the first section load eagerly; all others load lazily (automatic, based on DOM position). - Header and footer load in the lazy phase via `loadHeader()` / `loadFooter()`.

Key Files & How They Connect

File	Role	Connects To
<code>scripts/aem.js</code>	Core library — <code>loadBlock</code> , <code>loadCSS</code> , <code>decorateBlock</code> , <code>loadHeader</code> , <code>loadFooter</code> , <code>decorateSections</code> , <code>decorateBlocks</code> , <code>createOptimizedPicture</code> , etc. DO NOT MODIFY.	Imported by <code>scripts.js</code> and most blocks
<code>scripts/scripts.js</code>	Entry point — orchestrates E-L-D, plus a Dynamic Media / Scene7 auto-block , <code>decorateButtons()</code> , <code>decorateMain()</code> , and <code>moveInstrumentation()</code> (exported and used by nearly every card/carousel/columns block).	Imports from <code>aem.js</code> ; exported by blocks

File	Role	Connects To
scripts/delayed.js	Delayed phase target. Currently a stub (<code>// add delayed functionality here</code>) — no analytics/marketing tags are wired up yet.	Imported ~3s after load
styles/brand.css	University of Nottingham brand tokens (palette, type, CTA radius, heading scale). @imported at the top of styles.css.	Referenced by styles.css and all blocks
styles/styles.css	Critical CSS — semantic tokens, typography, section styles, and the full button system. Loaded eagerly.	Imports brand.css; referenced everywhere
styles/fonts.css	@font-face for circularxx, inter, domine, lora. Loaded lazily.	Font families referenced in brand.css
blocks/{name}/{name}.js + .css	Block logic + scoped styles. Auto-loaded when the block class appears in the DOM.	May import createOptimizedPicture from aem.js and moveInstrumentation from scripts.js
models/_*.json	Universal Editor component definitions/models/filters (partials) merged into aggregate files by <code>npm run build:json</code> .	Consumed by the authoring environment

Execution flow: page load → `scripts.js` → `loadEager()` (`decorateMain` + first section + eager fonts on desktop) → `loadLazy()` (remaining sections, header, footer, `fonts.css`, `lazy-styles.css`) → `loadDelayed()` (imports `delayed.js` after 3s).

The Dynamic Media / Scene7 Auto-Block (`scripts.js`)

A notable customisation in `scripts.js` is an excat-generated auto-block that converts Dynamic Media URLs into responsive `<picture>` elements. Key points for maintainers:

- `detectDynamicMediaUrl()` classifies a URL as scene7 (path starts with `/is/image/`) or dm-openapi (`delivery-p{n}-e{n}.adobecloud.com + /adobe/assets/urn:`). **Relative URLs are rejected up front** so ordinary site links are not misclassified.
- `buildScene7Rendition()` manipulates the query string by hand (never via `URLSearchParams`) because Scene7's IS/Image template syntax requires a literal `$` that would otherwise be percent-encoded and dropped.
- `buildDynamicMediaImages()` walks `<main>` for anchors carrying DM URLs (either in `href` for unlinked images, or in `title` for linked images — the markdown `[text](url "title")` round-trip) and rebuilds them as `<picture>`. It carefully strips any `button/button-container` classes that `decorateButtons()` may have added, and only unwraps `<p>` wrappers (never `<div>` block cells).
- `window.__dmRender__` is registered as a hook so that block decorators calling `createOptimizedPicture()` on DM URLs preserve their query parameters instead of having them stripped by the path-only optimizer in `aem.js`. Returns `null` for non-DM URLs so normal images fall through to standard optimization.

Button Decoration (`decorateButtons` in `scripts.js`)

Authored emphasis maps to button classes, then to brand styling in `styles.css`:

Authored formatting	Class added	Visual (styles.css)
strong link	.primary	Solid navy (<code>--text-color #10263b</code>), inverts to <code>--dark-color/--light-color</code> on hover
<i>*em*</i> link	.secondary	Transparent w/ navy border; cream fill on hover
strong + em link	.accent	Pink CTA — <code>--uon-pink #d7336c</code> , darkens to #b31e54 on hover, white text

All buttons use `border-radius: var(--cta-radius)` (**68px pill**). Only links with authored `strong/em` formatting are promoted to buttons; plain links and URL-display links are skipped. Note that the global `a: hover` rule keeps links at `--link-color` (navy) with an underline — this is source-faithful behaviour, deliberately not gold on hover for body links.

Local Development Setup

Prerequisites

- Node.js (LTS; no `.nvmrc` pinned — use the boilerplate default)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
git clone https://github.com/polizzigaetano/nottingham-ac
cd nottingham-ac
npm install
aem up --no-open --forward-browser-logs
```

Local Server — Repoless / Multi-Site

Because `polizzigaetano` is a repoless org hosting many sites, specify the site's preview URL when starting the local server so the AEM CLI resolves the correct content:

```
aem up --pages-url=https://main--nottingham-ac--polizzigaetano.aem.page
```

Without `--pages-url`, local preview may fail or show the wrong site's content.

- URL: `http://localhost:3000` — auto-reloads on file changes, serves your local working copy (even uncommitted code) with previewed content.
- Inspect delivered markup with `curl http://localhost:3000/path, .../path.md, or .../path.plain.html`.

Linting

```
npm run lint      # eslint (Airbnb) + stylelint
npm run lint:fix  # auto-fix
```

Regenerating Universal Editor Models

After editing any `models/_*.json` partial (or a block's `_{block}.json`), regenerate the aggregate files:

```
npm run build:json # rebuilds component-models / -definition / -filters .json
```

Design System

The tokens below were **reconciled against a Stardust computed-style extraction of the live nottingham.ac.uk site**. The source of truth is the artifact at `stardust/current/_brand-extraction.json` (per-element `getComputedStyle` aggregation across 5 template pages, colours frequency-ranked, `:root` custom properties read directly). The site's `--global*` variables map directly onto the brand tokens documented here.

Brand Tokens (`styles/brand.css`)

```
/* Fonts */
--heading-font-family: lora, domine, georgia, serif; /* headings render in Lora (serif) */
--body-font-family: circularxx, inter, arial, sans-serif; /* body in Circular */

/* Brand colours (mapped from site --global* vars) */
--uon-navy: #10263b; /* --globalBlue / --textColor: dominant bg + text */
--uon-navy-deep: #18194f; /* secondary dark: stat panels, overlays */
--uon-gold: #deb406; /* stat-panel accent */
--uon-pink: #d7336c; /* --globalPink: primary CTA on interior pages */
--uon-cream: #faf6ef; /* --globalBeige: section band */
--uon-grey: #f3f4f5; /* --globalGrey: light surface */

/* Extended brand accents (used sparingly) */
--uon-light-blue: #009bc1;
--uon-turquoise: #37b4b0;
--uon-purple: #792d85;
--uon-red: #b91c2e;

/* Semantic colours */
--background-color: #fff;
--text-color: #10263b;
--link-color: #10263b;
--link-hover-color: #deb406;

/* Layout */
--section-padding: 48px;
--nav-height: 86px;

/* Pill CTA radius (measured 68px primary, 54px secondary) */
--cta-radius: 68px;

/* Heading size scale (desktop) */
--heading-size-xxl: 56px; /* H1 */
--heading-size-xl-plus: 42px; /* large section H2 */
--heading-size-xl: 32px; /* H2 */
--heading-size-l: 28px;
--heading-size-m: 22px;
--heading-size-s: 18px;
--heading-size-xs: 16px;
```

`styles.css` layers semantic aliases on top: `--light-color: var(--uon-cream)`, `--dark-color: var(--uon-navy)`, plus mobile-first heading sizes that swap to the desktop scale above at the 900px breakpoint. Body sizes: `--body-font-size-m: 18px`, `-s: 16px`, `-xs: 14px`.

Fonts (styles/fonts.css)

Family	Weight	Format	File	Usage
circularxx	400	woff2	CircularXX-Regular.woff2	Body (primary)
inter	400	woff2	Inter-Regular.woff2	Body fallback
domine	400	woff2	Domine-Regular.woff2	Heading fallback
lora	400	woff2	Lora-Regular.woff2	Headings (primary)

All use font-display: swap. Loaded lazily (see E-L-D).

Breakpoints

Mobile-first with min-width queries. Note the codebase uses the modern range syntax @media (width >= 900px).

Name	Min-Width	Usage
Mobile	0	Default styles
Tablet	600px	@media (min-width: 600px)
Desktop	900px	@media (width >= 900px) — heading scale + eager fonts
Large	1200px	content max-width cap

Section Styles

Available via section metadata: - .section.light / .section.highlight — cream band (--uon-cream), full-bleed padding. - .section.dark — navy background (--uon-navy), white text; headings and links forced white. - Default-content is styled as a breadcrumb (chevron separators, teal links).

Blocks Reference

27 block folders exist under blocks/. Alongside the standard boilerplate blocks (cards, columns, header, footer, hero, fragment), the migration added a suite of custom, brand-styled blocks. Every custom block exports default function decorate(block) following the standard pattern: read authored rows, classify cells (image vs body via cell.querySelector('picture')), rebuild into a semantic list/structure, then re-optimize images with createOptimizedPicture. Cards/carousel/columns blocks call moveInstrumentation() (from scripts.js) to preserve Universal Editor authoring attributes on the rebuilt DOM.

Featured / highlighted blocks

hero-video-dark — Full-bleed dark media hero: poster image with an overlaid circular play button; clicking opens the video in a <dialog> modal. Key design points (from blocks/hero-video-dark/hero-video-dark.js): - **Lazy video, nothing autoplays on load.** The modal's player src is *not* set at build time. It is set only in openModal() and cleared in closeModal() (and on the native close/Escape event), so no video plays in the background. - **withAutoplay(url)** appends autoplay=1 for YouTube/Vimeo embed URLs so the video starts when the modal opens. - **File vs embed detection:** URLs matching \.(mp4|webm|ogg|m3u8) render into a <video controls playsinline preload="none"> element (played via media.play() with a caught promise); all others render into an <iframe> with allow="autoplay; fullscreen; picture-in-picture". - Renders just the poster (no play

button) when no video URL is authored. Poster uses `createOptimizedPicture(..., true, [{width: '2000'}])` (eager).

hero-card-dark — Full-bleed photographic banner with an opaque navy card overlapping the top-left, holding heading + short paragraph + single CTA. First picture-only cell becomes the background (eager optimized picture); remaining cells flow into a `.hero-card-dark-content` overlay.

cards-promo-dark-withimg-4 — Grid of dark promo tiles (typically 2×2): each tile = photo + heading + CTA (no body text). Rebuilds rows into a `<ul>`; classifies each cell as `-image` (single-picture cell) or `-body`; lazy-optimizes images at 750px width.

cards-promo-light-withimg-3 — Light sibling of the above (up to 3-up): photo + heading + optional paragraph + CTA. Same list/cell-classification/lazy-optimize pattern.

Full block inventory

Block	Purpose
<code>cards</code>	Boilerplate cards block
<code>cards-icon-light</code>	4-up credibility strip: small icon above a caption
<code>cards-logos-light</code>	Partner/accreditation logo strip. Keeps authored <code><picture></code> as-is (transparent PNGs; webp conversion would flatten transparency onto a black matte)
<code>cards-minimal-dark-withimg-5</code>	Row of image tiles, each a photo with an overlaid pill CTA
<code>cards-nav-light</code>	Navigation cards: image + title + caption, whole card is a link
<code>cards-promo-dark</code>	Text-only dark promo tiles: heading + one-line description + CTA
<code>cards-promo-dark-withimg-4</code>	Dark promo tiles with image (2×2) — <i>featured</i>
<code>cards-promo-light</code>	Text-only light promo cards
<code>cards-promo-light-withimg-3</code>	Light promo cards with image (3-up) — <i>featured</i>
<code>cards-ranking-gold</code>	Stat tiles: large stat figure + label + optional citation (first line = stat)
<code>cards-steps-light</code>	Numbered step rows (<code></code> , auto-incrementing): title + text + optional CTA
<code>carousel-events-light</code>	Rotating event cards: image + date + title + "Read more"; arrows/dots scroll
<code>carousel-hero-dark</code>	Full-bleed auto-cycling hero (6s autoplay) with prev/pause/next; slide = image + text panel
<code>carousel-minimal-dark-withimg-5</code>	Rotating news cards: image + headline + teaser + link
<code>columns</code>	Boilerplate columns block (2 commits — lightly customized)
<code>columns-minimal-dark-withimg-2</code>	Two-column dark feature banner: text col + large image col
<code>columns-video-light</code>	Two-column light section: text panel + embedded video (link → responsive iframe)
<code>columns-withimg-light</code>	Light sibling of <code>columns-minimal-dark-withimg-2</code>
<code>footer</code>	Site footer (customized)
<code>fragment</code>	Boilerplate fragment include
<code>header</code>	Site header / navigation (customized, 6 commits)
<code>hero</code>	Boilerplate hero

Block	Purpose
hero-card-dark	Photographic banner with overlapping navy card — <i>featured</i>
hero-minimal-dark-2	Full-bleed photo banner with overlaid heading + text + CTA
hero-video-dark	Dark media hero with lazy video modal — <i>featured</i>
nav-anchor-light	Light horizontal section-nav bar; active link gets <code>aria-current="page"</code>
search-course-light	"Find your course" panel: keyword input + study-level select + submit; levels/action authored

Universal Editor Models

Component definitions, models, and filters are authored as partials under `models/` (`_page.json`, `_section.json`, `_button.json`, `_image.json`, `_text.json`, `_title.json`, plus the aggregate partials `_component-definition.json`, `_component-models.json`, `_component-filters.json`). Individual blocks may also carry a `_{block}.json`. Run `npm run build:json` after any change to regenerate the top-level `component-definition.json`, `component-models.json`, and `component-filters.json` that the authoring environment consumes.

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/{name}.js` and `.css`.
2. Implement `export default function decorate(block)`. Inspect the delivered DOM first (`curl .../page.plain.html`) — do not assume structure; authors may omit or add cells.
3. Scope all CSS selectors to the block class (e.g. `._{name} .item`).
4. If it needs an authoring model, add `blocks/{name}/_{name}.json` and run `npm run build:json`.
5. Test locally, then `npm run lint`.

Modify Global Styles / Brand

1. Brand tokens live in `styles/brand.css`; semantic aliases and the button system in `styles.css`.
2. Prefer editing tokens over hard-coded values so blocks stay consistent.
3. Watch for CLS impact; keep critical CSS lean.

Add Analytics / Marketing Tool

1. Add to `scripts/delayed.js` (currently empty) — never to `scripts.js`.
2. Verify in the Network tab that it loads ~3s after page load.

Debug a Block

1. Check the browser console for decoration errors.
2. Compare expected authored structure vs the actual delivered DOM.
3. Confirm variant classes are applied and CSS specificity is correct.

Environments

Environment	URL Pattern	Purpose
Local	http://localhost:3000	Development
Feature Branch	https://{branch}-nottingham-ac-polizzigaetano.aem.page	PR testing
Preview	https://main-nottingham-ac-polizzigaetano.aem.page	Staging
Live	https://main-nottingham-ac-polizzigaetano.aem.live	Production

Git Workflow

- Branch naming: `feature/description` / `fix/description` (keep short — URL length limits).
- PR must include a feature-preview URL demonstrating the change, pass linting, and have no console errors.
- AEM Code Sync processes pushes automatically; a reviewer inspects the preview URL and merges to `main`.

Troubleshooting

Block not loading? Check the block folder name matches the class name; verify the JS default-exports `decorate`; check the Network tab for 404s on the block JS/CSS.

Styles not applying? Check specificity (all block CSS should be scoped to the block class); verify the file loaded; confirm `brand.css` tokens are resolving.

Dynamic Media images broken / stripped params? The DM auto-block in `scripts.js` and the `window.__dmRender__` hook handle Scene7/DM Open API URLs. If params vanish, confirm the URL passes `detectDynamicMediaUrl()` and that `createOptimizedPicture` is delegating to the hook.

Video autoplays or won't stop? In `hero-video-dark`, playback is controlled entirely by setting/clearing the media `src` in `openModal/closeModal`. Ensure any changes preserve the clear-on-close behaviour.

Content not updating? Clear browser cache; confirm the content was previewed in the author environment; allow 1–2 min for CDN cache.

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading (Keeping it 100)	https://www.aem.live/developer/keeping-it-100
Component Model Definitions	https://www.aem.live/developer/component-model-definitions

Resource	URL
Boilerplate (xwalk)	https://github.com/adobe-rnd/aem-boilerplate-xwalk